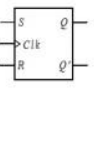
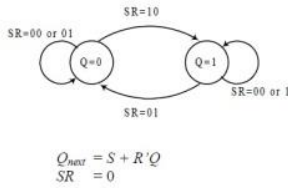
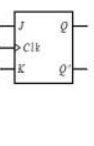
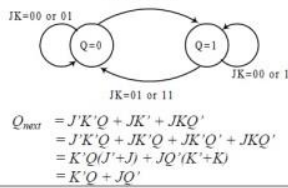
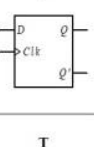
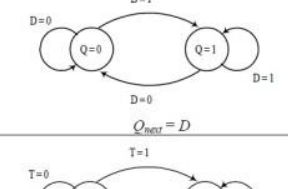
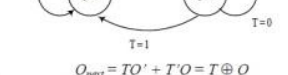


Name / Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table																																																								
SR 	<table><tr><th>S</th><th>R</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>×</td></tr><tr><td>1</td><td>1</td><td>1</td><td>×</td></tr></table>	S	R	Q	Q _{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	×	1	1	1	×	 SR=00 or 01 SR=01 SR=10 or 11 $Q_{next} = S + R'Q$ $SR = 0$	<table><tr><th>Q</th><th>Q_{next}</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>×</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>×</td><td>×</td></tr></table>	Q	Q _{next}	S	R	0	0	0	×	0	1	1	0	1	0	0	1	1	1	×	×
S	R	Q	Q _{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	×																																																								
1	1	1	×																																																								
Q	Q _{next}	S	R																																																								
0	0	0	×																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	1	×	×																																																								
JK 	<table><tr><th>J</th><th>K</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	J	K	Q	Q _{next}	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	0	1	0	0	1	1	0	1	1	1	1	0	1	1	1	1	0	 JK=00 or 01 JK=01 or 11 JK=10 or 11 $Q_{next} = J'K'Q + JK' + JQ$ $= J'K'Q + JK'Q + JK'Q' + JQ$ $= K'Q(J' + J) + JQ'(K' + K)$ $= K'Q + JQ'$	<table><tr><th>Q</th><th>Q_{next}</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>×</td></tr><tr><td>0</td><td>1</td><td>1</td><td>×</td></tr><tr><td>1</td><td>0</td><td>×</td><td>1</td></tr><tr><td>1</td><td>1</td><td>×</td><td>0</td></tr></table>	Q	Q _{next}	J	K	0	0	0	×	0	1	1	×	1	0	×	1	1	1	×	0
J	K	Q	Q _{next}																																																								
0	0	0	0																																																								
0	0	1	1																																																								
0	1	0	0																																																								
0	1	1	0																																																								
1	0	0	1																																																								
1	0	1	1																																																								
1	1	0	1																																																								
1	1	1	0																																																								
Q	Q _{next}	J	K																																																								
0	0	0	×																																																								
0	1	1	×																																																								
1	0	×	1																																																								
1	1	×	0																																																								
D 	<table><tr><th>D</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>×</td><td>0</td></tr><tr><td>1</td><td>×</td><td>1</td></tr></table>	D	Q	Q _{next}	0	×	0	1	×	1	 D=0 D=1 $Q_{next} = D$	<table><tr><th>Q</th><th>Q_{next}</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q _{next}	D	0	0	0	0	1	1	1	0	0	1	1	1																																
D	Q	Q _{next}																																																									
0	×	0																																																									
1	×	1																																																									
Q	Q _{next}	D																																																									
0	0	0																																																									
0	1	1																																																									
1	0	0																																																									
1	1	1																																																									
T 	<table><tr><th>T</th><th>Q</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	T	Q	Q _{next}	0	0	0	0	1	1	1	0	1	1	1	0	 T=0 T=1 $Q_{next} = TQ' + T'Q = T \oplus Q$	<table><tr><th>Q</th><th>Q_{next}</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q _{next}	T	0	0	0	0	1	1	1	0	1	1	1	0																										
T	Q	Q _{next}																																																									
0	0	0																																																									
0	1	1																																																									
1	0	1																																																									
1	1	0																																																									
Q	Q _{next}	T																																																									
0	0	0																																																									
0	1	1																																																									
1	0	1																																																									
1	1	0																																																									

Theorems of Boolean Algebra

#	Theorem		
1	$A + A = A$	$A \cdot A = A$	Tautology Law
2	$A + 1 = 1$	$A \cdot 0 = 0$	Union Law
3	$\overline{(\overline{A})} = A$		Involution Law
4	$A + (B \cdot C) = (A + B) \cdot C$	$A \cdot (B + C) = (A \cdot B) + C$	Associative Law
5	$A + \overline{A} \cdot B = A + B$	$\overline{A} \cdot \overline{B} = \overline{A + B}$	De Morgan's Law
6	$A + A \cdot B = A$	$A \cdot (A + B) = A$	Absorption Law
7	$A + \overline{A} \cdot B = A + B$	$A \cdot (\overline{A} + B) = A \cdot B$	
8	$AB + A\overline{B} = A$	$(A + B)(A + \overline{B}) = A$	Logical adjacency
9	$AB + \overline{A}C + BC = AB + \overline{A}C$	$(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)$	Consensus Law

Duality (OR and AND, 0 and 1 can be interchanged)

Postulates of Boolean Algebra – cont.

4. Distributive Law

- $x \cdot (y + z) = x \cdot y + x \cdot z$ (over +)
- $x + (y \cdot z) = (x + y) \cdot (x + z)$ (+ over ·)

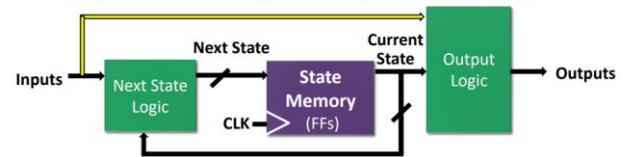
5. For every element x in the set B , there exists an element $\bar{x}$ in the set B , such that

- $x + \bar{x} = 1$
 - $x \cdot \bar{x} = 0$
- ($\bar{x}$ is called the **complement** of x)

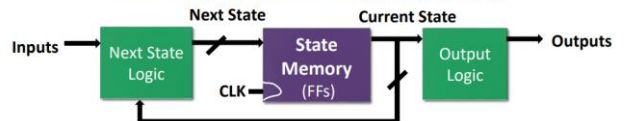
6. There exist at least two distinct elements in the set B

Moore vs Mealy FSMs : Different Output Generation

- Mealy machine: output is a function of a **present state & inputs**.



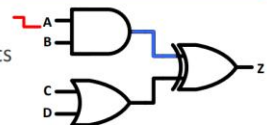
- Moore machine: output is a function of a **present state only**.



Sequential Logic Circuits?

Combinational Logic Circuits:

- Outputs depend on current inputs

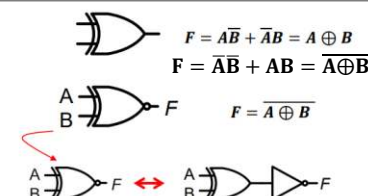


Sequential Logic Circuits:

- Outputs depend on **current and previous** inputs → Memory!
- Requires separation of previous, current, future : **states**
- 2 Types of sequential circuits:

Synchronous	Asynchronous
Clocked: need a clock input	Unclocked
Responds to inputs at discrete time instants governed by a clock input	Responds whenever input signals change

XOR and XNOR Gates



Truth Table (XOR, XNOR):

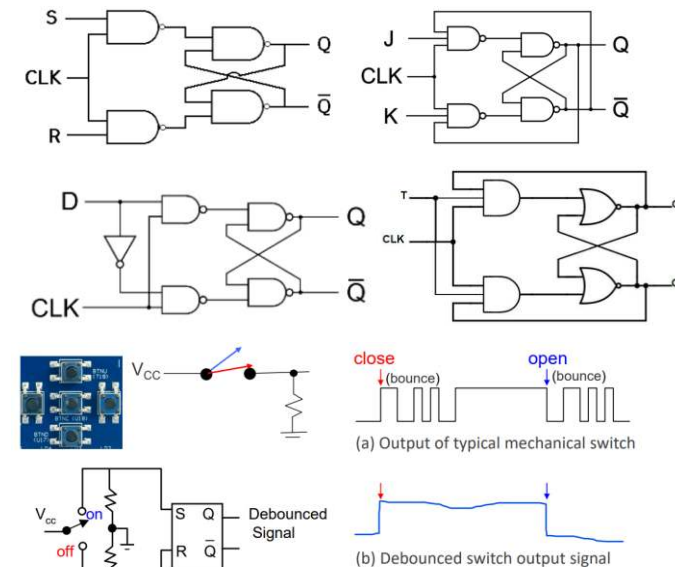
A	B	A ⊕ B	A ⊙ B
0	0	0	1
1	0	1	0
0	1	1	0
1	1	0	1

XOR

- F is TRUE if $A \neq B$

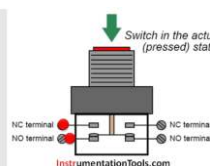
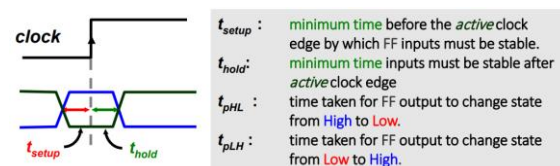
XNOR

- F is TRUE if $A = B$



S	R	Output
0	0	Hold
0	1	Q = 0
1	0	Q = 1
1	1	Invalid
0 0 is the rest state		

- Mechanical switches bounce before settling down which may cause problems as inputs.
- Switch **debouncing** is a common use of S-R FFs.



A FF has the following timing parameters :

$t_{pHL} = t_{pLH} = 50ns$
 t_{setup} and t_{hold} = negligible (smaller than 0.01ns)

What is the **theoretical limit** of the maximum clock frequency (in MHz) that I can apply to this FF?
Please type in only the number in MHz in the blank below :

General Feedback

The FF has a output propagation delay of 50ns (either for signal going from low-to-high, or high-to-low). This means that the output Q takes 50ns to change its value.

Thus, the maximum theoretical clock frequency that can be applied is $1/50ns = 20MHz$.

In practice, we would need a clock frequency that is slightly lower than 20MHz for us to observe the output Q changing. However, this 20MHz is the theoretical limit of the operating clock frequency for the FF.

Precedence	Operator	Description	Examples: a = 4'b1010, b = 4'b0000
High	!, ~	Logical negation, Bit-wise NOT	!a = 0, !b = 1, ~a = 4'b0101, ~b = 4'b1111
	&, , ^	Reduction (Outputs 1-bit)	&a = 0, a = 1, ^a = 0
	{_, _}	Concatenation	{b, a} = 8'b00001010
	{n{ _ }}	Replication	{2{a}} = 8'b10101010
	*, /, %,	Multiply, *Divide, *Modulus	3 % 2 = 1, 16 % 4 = 0
	+, -	Binary addition, subtraction	a + b = 4'b1010
	<<, >>	Shift Zeros in Left / Right	a << 1 = 4'b0100, a >> 2 = 4'b0010
	<=, <, >, >=	Logical Relative (1-bit output)	(a > b) = 1
	==, !=	Logical Equality (1-bit output)	(a == b) = 0 (a != b) = 1
	&, ^,	Bit-wise AND, XOR, OR	a & b = 4'b0000 a b = 4'b1010
	&&,	Logical AND, OR (1-bit output)	a && b = 0 a b = 1
Low	?:	Conditional Operator	<out> = <condition> ? if_ONE : if_ZERO

Blocking & Non-blocking

Verilog supports two types of assignments within

- always**
- = blocking assignment
 - o Sequential evaluation
 - o Immediate assignment
- <=** non-blocking assignment
- o Sequential evaluation
 - o Deferred assignment

always @ (*)

begin

x = y; 1) Evaluate y, assign result to x

z = ~x; 2) Evaluate ~x, assign result to z

end

always @ (*)

begin

x <= y; 1) Evaluate y, defer assignment

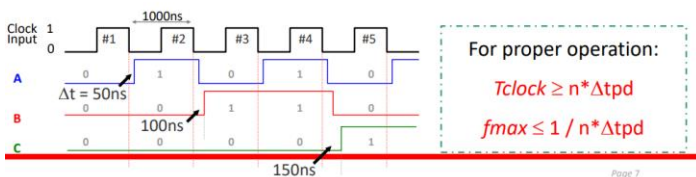
z <= ~x; 2) Evaluate ~x, defer assignment

end 3) Assign x and z with new values

Behaviour	x	y	z	Deferred
Initial Condition	0	0	1	
y changes	0	1	1	
x = y	1	1	1	x <= 1
z = ~x	1	1	0	z <= 1
Assignment	1	1	1	

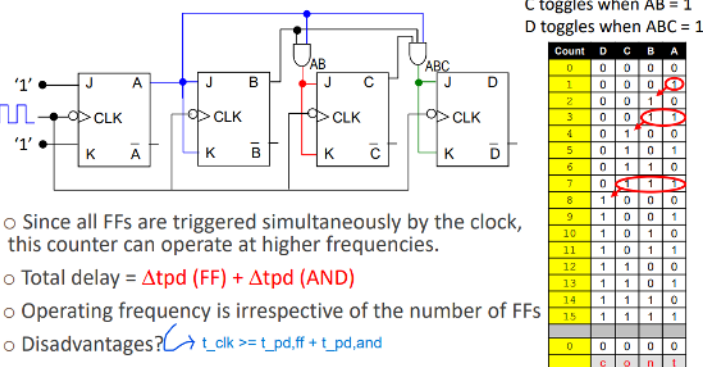
t_{pd} → limiting frequency

- o Ripple counters are very easy to implement
 - o But they have a major drawback → they cannot operate beyond a **limiting frequency**
 - o Due to **propagation delays** of the FFs in the chain **adding up**:
1. Clock input FF₁: t₀
 2. Clock input FF₂: t₀ + Δt_{pd}
 3. Clock input FF₃: t₀ + 2*Δt_{pd}
 4. Clock input FF_N: t₀ + (n - 1) * Δt_{pd}
- the nth FF changes state at n * Δt_{pd} after t₀



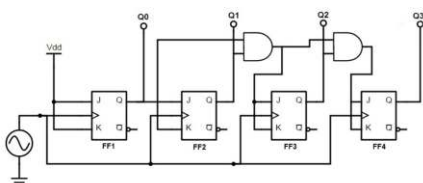
Synchronous (Parallel) Counters

Mod-16 Synchronous Parallel Counter :

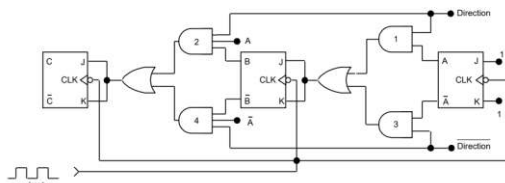


- o Since all FFs are triggered simultaneously by the clock, this counter can operate at higher frequencies.
- o Total delay = Δt_{pd} (FF) + Δt_{pd} (AND)
- o Operating frequency is irrespective of the number of FFs
- o Disadvantages: t_{clk} >= t_{pd,ff} + t_{pd,and}

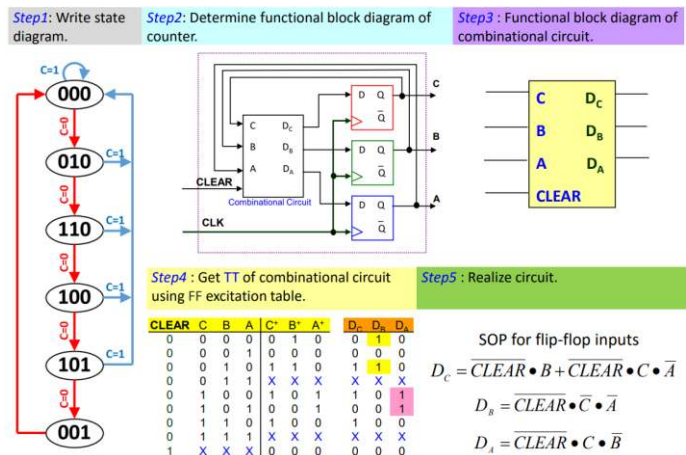
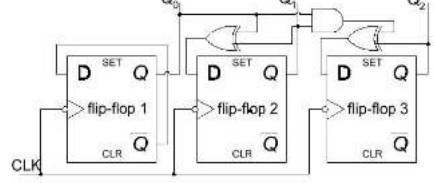
Mod-16 Count Down



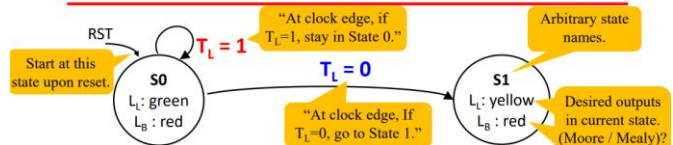
Up/Down Synchronous Counter



Mod-8 Count Up D-FF



State Transition Diagrams



- o Circles represent **states**.
* Each state specifies values for **all outputs** (Moore)
- o Arcs represent **transitions** between states.
* Labels → input that **triggers** the transition.
* Transitions take place on the **active edge** of the clock.
- o Arc from outer space indicates initial state upon reset
- o Within each state, for any combination of input values, there's exactly **one applicable** arc.

FSM in Verilog

```

module fsm(input clk, ..., output ...);
    reg __state, nextstate;

    parameter S0 = 2'b00;
    parameter S1 = 2'b01;

    always @ (*)
    case (state)
        S0 : nextstate = S1;
        S1 : nextstate = S0;
    endcase

    always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else state <= nextstate;

    assign y = (state == S0);
endmodule

```

parameter is used to define constants within a module, improving code readability.

Next State Combinational Logic:
(*) code is triggered whenever any input changes → combinational logic.
case represents next state table.

Sequential Logic:
Use <= to infer flip-flops.
Is this Sync or Async reset?

Output Logic: Use **assign** to infer combinational logic.

Equality Comparison:
a == b evaluates to 1 if a equals b.

```

module counter(input clear, clk, output reg [3:0] q);

always @ (posedge clk) begin
    q <= clear ? (q - 1) : 4'b0000;
end

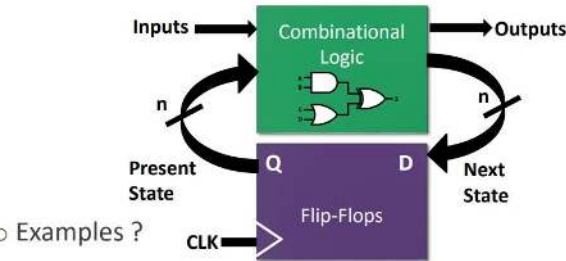
endmodule

```

- o What counter does this code describe?
- 1. Positive/Negative Edge clock triggered?
- 2. Asynchronous / Synchronous Clear?
- 3. Count Up / Down Counter?

What is a FSM?

- FSM : Finite State Machine
- Synchronous Machine with “states” of operation
- At each **active clock edge**, combinational logic computes **outputs and next state**, as a function of **inputs and present state**



○ Examples ?

Structure

State Memory

- set of n FFs store current state of machine; up to 2ⁿ states.
- FFs can be J-K or D, but D FFs simpler (1 input vs. 2 inputs for J-K FFs)

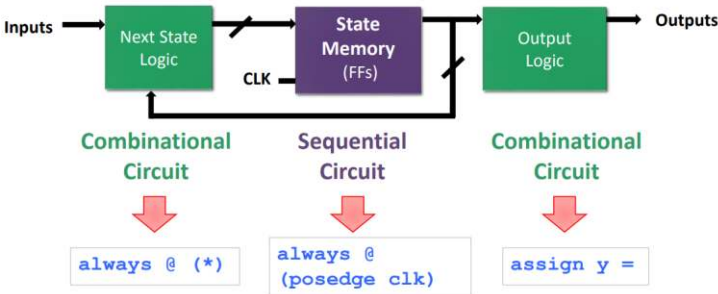
Next State Logic

- combinational circuit which decides the next state of the machine based on current state and inputs:
 $Next\ state = f(inputs, current\ state)$

Output logic

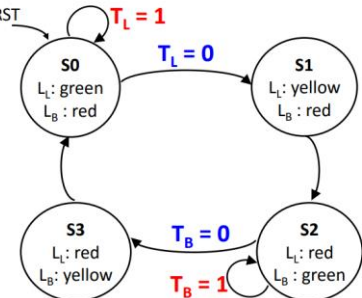
- combinational circuit which creates the output
- Moore : outputs depend on current state
 $outputs = g(current\ state)$
- Mealy : outputs depend on the current state & inputs
 $outputs = g(inputs, current\ state)$

Verilog~! – Code Structure



Step 1 : State Transition Diagram

- Block Diagram of Desired System
- Design **State Transition Diagram** to represent FSM



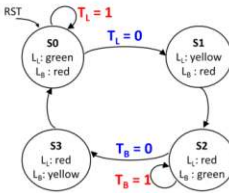
Step 2 : Next State Table

1. From STD, find the number of states
2. Number of bits / FFs required = ? to go through these states
3. State Assignment

State	S ₁ S ₀
S0	00
S1	01
S2	10
S3	11

4. Next State Table

Current State	Inputs		Next State
S	T _L	T _B	S+
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0



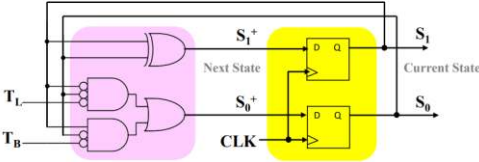
Step 2 : Next State Table

5. State Generator Circuit

Current State		Inputs		Next State	
S ₁	S ₀	T _L	T _B	S ₁ ⁺	S ₀ ⁺
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

$$S_1^+ = \overline{S_1} \overline{S_0} + S_1 \overline{S_0} \overline{T_B} + S_1 \overline{S_0} T_B$$
$$= S_1 \oplus S_0$$

$$S_0^+ = \overline{S_1} \overline{S_0} \overline{T_L} + S_1 \overline{S_0} T_B$$



Step 3 : Output Logic

1. Output Truth Table

Current State		Outputs			
S ₁	S ₀	L _{L,1}	L _{L,0}	L _{B,1}	L _{B,0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Output	L _L L ₀
Green	00
Yellow	01
Red	10

$$L_{L,1} = S_1 \quad L_{L,0} = \overline{S_1} \overline{S_0}$$
$$L_{B,1} = \overline{S_1} \quad L_{B,0} = S_1 S_0$$

